APPENDIX B
ARTIFACT APPENDIX

A. Description & Requirements

1) How to access: Source code, scripts, configurations, the Naz-mix ($k = 1$–4) LoRA checkpoint, and three small demonstration datasets: https://github.com/cssnku/CSSLabNKU. The artifact implements behavioral analysis, LoRA training with Poly-1 loss, and password generation with dynamic beam search. The instructions below describe how to install the software and run these components on your own hardware. Data requirements and restrictions are described under Benchmarks.

2) Hardware dependencies:

- Linux (x86-64), 8+ CPU cores, 16+ GB RAM.
- One NVIDIA GPU with $\geq$16 GB VRAM and bf16 support (e.g., RTX 3090/4090, A10G, L4) is required for model evaluation and optional training. On a single RTX 4090, the bundled mini training and evaluation typically complete within 10 minutes in total, depending on the environment. These times exclude installation and model download. For the full-scale Scenario #5, Table V of the paper reports 1 h 35 min for training and 9 h 27 min for testing.
- The behavior-metrics experiment (E1) runs on CPU only and completes within minutes.

3) Software dependencies: Python 3.10+; CUDA-capable PyTorch $\geq$2.1; transformers $\geq$4.46,$<$ 4.47; peft $\geq$0.13,$<$ 0.15; datasets $\geq$2.19; numpy; pandas; tqdm; huggingface_hub; matplotlib; tensorboard $\geq$2.14; accelerate $\geq$0.26; safetensors. TensorBoard is required by the training logger; accelerate and safetensors support automatic device placement and checkpoint handling. A pinned dependency list is in `requirements.txt`. Network access is required once to download the base model Qwen/Qwen2.5-0.5B (revision 060db6499f32faf8b98477b0a26969ef7d8b9987, $\sim$1 GB) from Hugging Face.

4) Benchmarks: The mini workflow uses a small de-identified subset of Scenario #5 (Naz corpus, $k \in [1, 4]$): 2,000 training and 201 test samples, each a record with a space-joined `Old password` field and a `password` target field. The artifact provides three small demonstration datasets for the mini workflow. These data contain passwords only, without email addresses or account identifiers. The full COMB/Cit0day/Naz corpora are also not redistributed, consistent with the paper's ethics statement. To run the full-scale configurations in `configs/full/`, obtain the underlying corpora subject to their applicable access conditions and construct the data independently.

B. Artifact Installation & Configuration

Create and activate a Python 3.10 Conda environment, then run the following commands from the extracted repository root. Lines ending with a backslash are shell continuations. The GPU check must print `True` before E2/E3.

```
conda create -n polyguess python=3.10 -y
conda activate polyguess
```

```
pip install torch --index-url \
  https://download.pytorch.org/whl/cu124
pip install -r requirements.txt
pip install "tensorboard>=2.14" \
  "accelerate>=0.26" safetensors
```

```
bash scripts/download_models.sh
sha256sum model/Qwen0.5B/model.safetensors
python - <<'PY'
import torch
print(torch.cuda.is_available())
PY
```

Base model: `model/Qwen0.5B`.

LoRA adapter: `checkpoints/naz_1_4_final`.

All paths are relative to the repository root. Use the base Qwen/Qwen2.5-0.5B model, not its Instruct variant. The SHA-256 of `model.safetensors` must match the expected value printed by the download script. If an older model directory is present or the checksum differs, move that directory aside and rerun the download script before evaluation; the Instruct variant has incompatible EOS-token settings and can produce empty candidate lists.

C. Experiment Workflow

Run E1 independently on CPU to compute the nine behavioral metrics. E2 optionally trains a LoRA adapter with Poly-1 loss. E3 generates password guesses using the supplied checkpoint and can be run without E2. Both E2 and E3 require a compatible GPU. The commands and expected outputs are listed below; `README.md` provides sample outputs and troubleshooting.

D. Major Claims

- **(C1):** The artifact implements the nine behavioral metrics used in Sec. III to characterize multi-password reuse, including global diversity, local similarity, and sparse decisive transformations. Experiment (E1) exercises this analysis pipeline on the bundled mini subset and produces the corresponding metric outputs.
- **(C2):** POLYGUESS implements the proposed Poly-1 loss, which introduces a confidence-dependent correction to the learning signal (Sec. IV-C, Eq. 9). Experiment (E2) demonstrates that the implementation can be trained end-to-end on the bundled mini subset.
- **(C3):** POLYGUESS implements the end-to-end multi-password targeted guessing pipeline, including the fine-tuned LoRA model and dynamic beam search. Experiment (E3) exercises this pipeline on the Scenario #5 mini subset using the provided checkpoint.

These reduced-scale experiments exercise the analysis, training, and generation pipelines; their numerical outputs apply only to the mini subset. Reproducing the paper's full-scale results requires the original data preparation, full-scale configurations, and longer runs (see Table V for Scenario #5 timings).

E. Evaluation

The following experiments use the mini subset. For each experiment, check the stated output files and success criteria after execution. Runtime estimates exclude dependency installation and model download; actual times depend on the hardware and environment.

1) Experiment (E1): Nine-metric behavior analysis:

[5–10 human-minutes; <10 compute-minutes on CPU]

[How to] The analysis script reads `data/mini/naz_mini_users.txt` (one password set per user per line) and computes the nine metrics (MeanSim, MaxSim, VarSim, PivotSim, TemplateCons, LenCV, MaxLCSCov, MS_PosCopyRate, MutRatio) grouped by password-count bin. In the paper's characterization, m denotes the total distinct-password count in a user's set, whereas k denotes the number of available sister passwords in an attack scenario.

[Preparation] `requirements.txt` installed.

[Execution]

```
python scripts/run_metrics.py
```

[Results] `results/metrics/Similarity_*.pdf` (nine figures) and per-bin percentages printed to stdout.

Success criteria. Exit code 0 and all nine figures produced.

2) Experiment (E2): Mini training with Poly-1 loss:

[optional; E2+E3 together complete within 10 minutes on a single RTX 4090]

[How to] The training script trains POLYGUESS (LoRA + Poly-1 loss) from the base model on `data/mini/naz_train_mini.json` (1 epoch, small batch, gradient accumulation).

[Preparation] Base model downloaded; GPU available.

[Execution]

```
bash scripts/run_mini_train.sh
```

[Results] `checkpoints/mini_train_output/` with the adapter and a decreasing training loss in the logs.

Success criteria. Exit code 0 and a checkpoint produced. This supports claim C2 by showing that the Poly-1 objective is implemented and trainable.

3) Experiment (E3): Mini evaluation with the provided checkpoint:

[E2+E3 together complete within 10 minutes on a single RTX 4090]

[How to] The evaluation script performs dynamic beam search on `data/mini/naz_test_mini.json` using the fine-tuned LoRA checkpoint and records the top-100 guesses per target.

[Preparation] Base-model checksum verified; GPU available. Use the supplied `checkpoints/naz_1_4_final` adapter; E3 does not require E2's newly trained adapter. Move any previous `results/mini_eval/` output aside before rerunning.

[Execution]

```
bash scripts/run_mini_eval.sh
```

[Results] Outputs are saved under `results/mini_eval/`:

- `input_output.jsonl`: one record per test target, containing top-100 guesses.
- `log.txt`: cumulative success counts at guess numbers [1,10,50,100].
- `eval_config.json`: the evaluation configuration.

Success criteria. Exit code 0; exactly 201 output records, each with a non-empty `outputPasswords` list; and a final log entry (index 200) with cumulative counts at [1,10,50,100]. Compare records and final counts with the reference examples in `README.md`. These checks support C3 (end-to-end pipeline functionality).

Troubleshooting. Empty candidate lists fail the success check. Verify the base-model variant and checksum before rerunning.

F. Customization

Experiment parameters (beam width list, batch size, max guesses, data paths) are set in `configs/mini/eval_naz_mini.ini` and `configs/mini/train_naz_mini.ini`. The metrics script accepts `-input`, `-output-prefix`, and `-sample-size`. Full-scale configurations are provided in `configs/full/` for reference only; the full-scale data are not redistributed for ethical reasons.

G. Notes

- Baseline models (e.g., PassLLM-II) are third-party implementations and are not bundled; the paper's Table VI reports their full-scale numbers for reference.
- The code does not embed analytics or tracking.